

Experiment/Case Study No : 4

Aim:

Use function types, scope, and closures; apply try catch; build a palindrome checker.

Software/Tools Required

- Visual Studio Code
- Google Chrome (Web Browser)

Experiment Program Code

1. palindromef.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Palindrome Checker</title>
</head>
<body>
  <link rel="stylesheet" href="pal.css">
  <h2>Palindrome Checker</h2>
  <input type="text" id="text-input" placeholder="Enter text">
  <button id="check-btn">Check</button>
  <p id="result"></p>

  <script src="pal.js"></script>
  <div style="position: fixed; bottom: 10px; right: 10px; font-family: sans-serif; font-size: 14px; color: #333; background-color: rgba(255,255,255,0.8); padding: 5px 10px; border-radius: 4px; z-index: 1000;">
    Name: M HUSSAIN KHAN <br>
    PRN: 8482871166
  </div>
</body>
</html>
```

2. pal.js

```
document.getElementById('check-btn').addEventListener('click', function() {
  const input = document.getElementById('text-input').value;
  const result = document.getElementById('result');

  try {
    const cleaned = input.toLowerCase().replace(/[^a-z0-9]/g, "");

    if (!cleaned) {
      throw new Error("Please enter at least one letter or number.");
    }

    const reversed = cleaned.split("").reverse().join("");

    if (cleaned === reversed) {
      result.textContent = "Yes , it is a palindrome.";
    } else {
      result.textContent = "No , it is not a palindrome.";
    }
  } catch (error) {
    result.textContent = error.message;
  }
});
```

3. pal.css

```
body {
  font-family: sans-serif;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  height: 100vh;
  margin: 0;
  background-color: #f0f2f5;
}

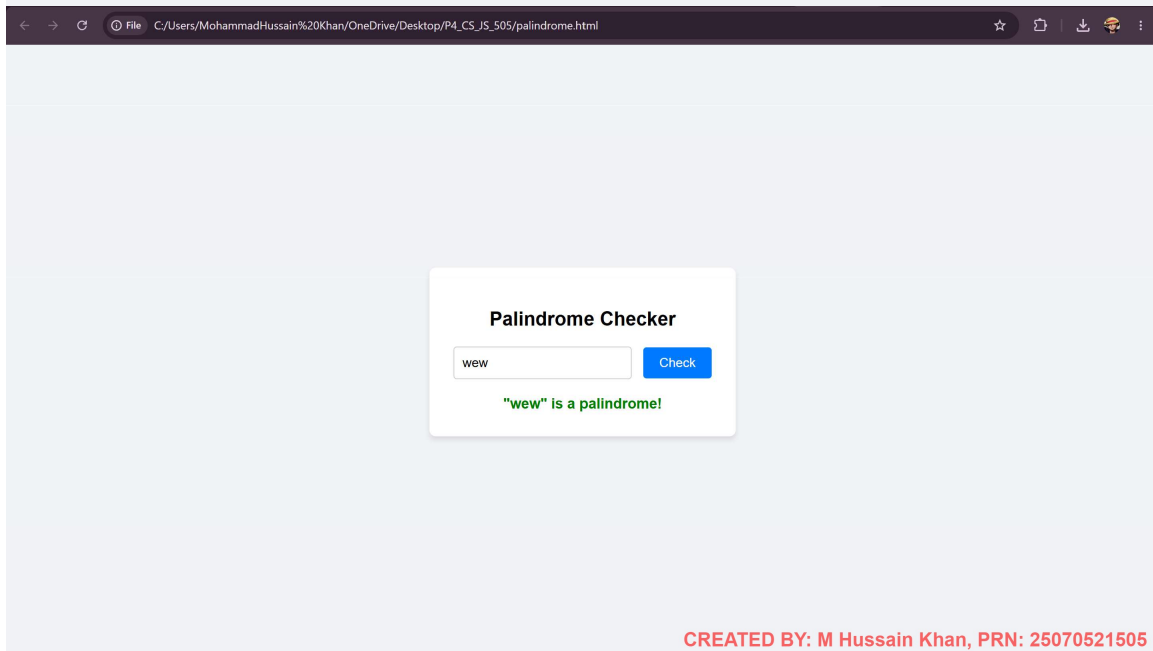
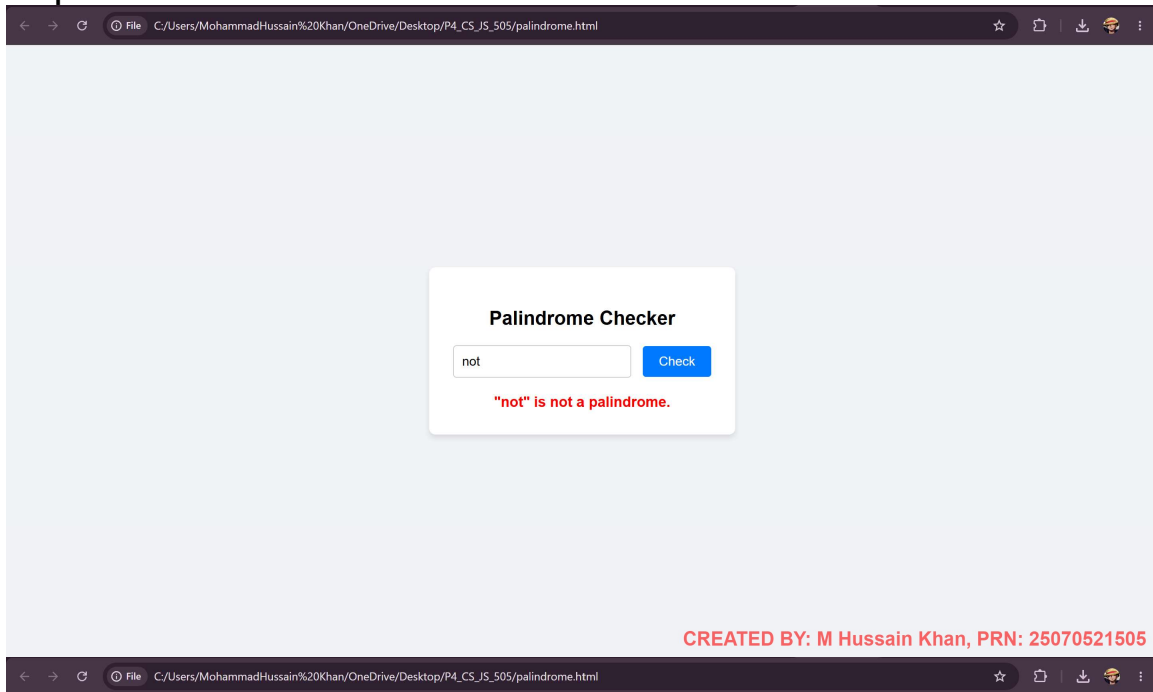
input {
  padding: 10px;
  font-size: 16px;
  border: 1px solid #ccc;
  border-radius: 4px;
}
```

```
button {  
  padding: 10px 20px;  
  font-size: 16px;  
  background-color: #007bff;  
  color: white;  
  border: none;  
  border-radius: 4px;  
  cursor: pointer;  
}
```

```
button:hover {  
  background-color: #0056b3;  
}
```

```
#result {  
  margin-top: 20px;  
  font-weight: bold;
```


Output



Case Study Title

Vehicle number plate valid number or not checker

Case Study Program Code

1. Valid_cs.html

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Vehicle Registration Validation</title>

  <link rel="stylesheet" href="Valid_cs.css">

</head>

<body>

  <div class="container">

    <h2>Vehicle Registration Check</h2>

    <form id="vehicleForm">

      <div class="input-group">

        <input type="text" id="regNumber" name="regNumber" placeholder=" "
maxlength="10" autocomplete="off">

        <label for="regNumber">Enter Registration Number</label>

      </div>

      <div class="rules-box">

        <h4>Validation Rules:</h4>

        <ul id="rulesList">
```

```
        </ul>
    </div>

    <div id="errorMessage" class="message error"></div>
    <div id="successMessage" class="message success"></div>

    <button type="submit">Validate Number</button>
</form>
</div>
<script src="Valid_cs.js"></script>
<div style="position: fixed; bottom: 10px; right: 15px; color: rgba(255, 0, 0, 0.6); font-size: 24px; font-weight: bold; z-index: 1000;">
    CREATED BY: M Hussain Khan,
    PRN: 25070521505
</div>
</body>
</html>
```

2. Valid_cs.css

```
:root {

    --neon-cyan: #00F0FF;
    --electric-pink: #FF0099;
    --vivid-green: #00FF94;
    --deep-space: #0F172A;
    --pure-white: #FFFFFF;
    --glass-bg: rgba(255, 255, 255, 0.75);
    --glass-border: rgba(255, 255, 255, 0.9);
```

```
--soft-shadow: 0 8px 32px 0 rgba(31, 38, 135, 0.15);  
  
}  
  
body {  
  font-family: 'Segoe UI', Roboto, Helvetica, Arial, sans-serif;  
  
  background: linear-gradient(135deg, #e0f7fa 0%, #fff0f5 50%, #e0fff4 100%);  
  background-size: 400% 400%;  
  animation: gradientBG 15s ease infinite;  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  min-height: 100vh;  
  margin: 0;  
  color: var(--deep-space);  
}  
  
@keyframes gradientBG {  
  0% { background-position: 0% 50%; }  
  50% { background-position: 100% 50%; }  
  100% { background-position: 0% 50%; }  
}  
  
.container {  
  
  background: var(--glass-bg);  
  backdrop-filter: blur(16px);  
  -webkit-backdrop-filter: blur(16px);
```



```
padding: 50px 40px;

border-radius: 24px;

border: 1px solid var(--glass-border);

box-shadow: var(--soft-shadow);

width: 100%;

max-width: 420px;

position: relative;

overflow: hidden;

animation: fadeIn 0.8s ease-out;

}
```

```
.container::before {

  content: "";

  position: absolute;

  top: -50px;

  right: -50px;

  width: 150px;

  height: 150px;

  background: var(--electric-pink);

  filter: blur(60px);

  opacity: 0.4;

  border-radius: 50%;

  z-index: -1;

}
```

```
.container::after {

  content: "";
```

```
position: absolute;

bottom: -50px;

left: -50px;

width: 150px;

height: 150px;

background: var(--neon-cyan);

filter: blur(60px);

opacity: 0.4;

border-radius: 50%;

z-index: -1;

}
```

```
h2 {

text-align: center;

background: linear-gradient(90deg, var(--electric-pink), var(--neon-cyan));

-webkit-background-clip: text;

-webkit-text-fill-color: transparent;

margin: 0 0 40px 0;

font-weight: 800;

letter-spacing: 1px;

text-transform: uppercase;

font-size: 26px;

}
```

```
.input-group {

position: relative;

margin-bottom: 30px;

}
```

```
input {  
  width: 100%;  
  padding: 16px 20px;  
  border: 2px solid transparent;  
  background: rgba(255, 255, 255, 0.6);  
  border-radius: 16px;  
  font-size: 16px;  
  color: var(--deep-space);  
  text-transform: uppercase;  
  outline: none;  
  transition: all 0.3s ease;  
  text-align: center;  
  font-weight: 700;  
  box-sizing: border-box;  
  box-shadow: inset 2px 2px 5px rgba(0,0,0,0.03);  
}
```

```
input:focus {  
  background: var(--pure-white);  
  border-color: var(--neon-cyan);  
  box-shadow: 0 0 15px rgba(0, 240, 255, 0.3);  
  transform: translateY(-2px);  
}
```

```
label {  
  position: absolute;  
  left: 20px;
```

```
top: 50%;  
transform: translateY(-50%);  
color: #8898aa;  
font-size: 14px;  
pointer-events: none;  
transition: 0.3s ease all;  
text-transform: uppercase;  
font-weight: 600;  
letter-spacing: 0.5px;  
background: transparent;  
padding: 0 4px;  
}
```

```
input:focus + label,  
input:not(:placeholder-shown) + label {  
    top: 0;  
    font-size: 11px;  
    color: var(--electric-pink);  
    background: var(--pure-white);  
    border-radius: 4px;  
    font-weight: 700;  
}
```

```
.rules-box {  
    background: rgba(255, 255, 255, 0.5);  
    border-left: 4px solid var(--vivid-green);  
    padding: 20px;  
    margin-bottom: 25px;
```

```
font-size: 13px;

color: #555;

line-height: 1.8;

border-radius: 12px;

box-shadow: inset 0 2px 5px rgba(0,0,0,0.02);

}
```

```
.rules-box h4 {

    margin: 0 0 12px 0;

    font-size: 12px;

    color: var(--vivid-green);

    text-transform: uppercase;

    letter-spacing: 1.5px;

    font-weight: 800;

    text-shadow: 0 0 10px rgba(0, 255, 148, 0.3);

}
```

```
.rules-box ul {

    margin: 0;

    padding-left: 20px;

    list-style-type: none;

}
```

```
.rules-box li {

    margin-bottom: 8px;

    position: relative;

}
```

```
.rules-box li::before {  
  content: "•";  
  color: var(--electric-pink);  
  font-weight: bold;  
  display: inline-block;  
  width: 1em;  
  margin-left: -1em;  
}
```

```
.rules-box li strong {  
  color: var(--deep-space);  
  font-weight: 700;  
}
```

```
.message {  
  padding: 15px;  
  margin-bottom: 20px;  
  font-size: 14px;  
  font-weight: 700;  
  display: none;  
  text-align: center;  
  letter-spacing: 0.5px;  
  animation: slideDown 0.4s ease;  
  border-radius: 12px;  
  border: 1px solid transparent;  
}
```

```
.error {
```

```
background: rgba(255, 0, 153, 0.1);  
color: var(--electric-pink);  
border-color: rgba(255, 0, 153, 0.3);  
box-shadow: 0 4px 15px rgba(255, 0, 153, 0.15);  
}
```

```
.success {  
  background: rgba(0, 255, 148, 0.15);  
  color: #00cc76;  
  border-color: rgba(0, 255, 148, 0.4);  
  box-shadow: 0 4px 15px rgba(0, 255, 148, 0.2);  
}
```

```
button {  
  width: 100%;  
  padding: 18px;  
  background: linear-gradient(90deg, var(--neon-cyan), var(--vivid-green));  
  color: var(--deep-space);  
  border: none;  
  border-radius: 16px;  
  font-size: 15px;  
  font-weight: 800;  
  cursor: pointer;  
  transition: all 0.3s ease;  
  text-transform: uppercase;  
  letter-spacing: 1.5px;  
  position: relative;  
  overflow: hidden;
```

```
    box-shadow: 0 10px 20px rgba(0, 240, 255, 0.25);  
}
```

```
button:hover {  
    transform: translateY(-3px);  
    box-shadow: 0 15px 30px rgba(0, 255, 148, 0.35);  
    filter: brightness(1.05);  
}
```

```
button:active {  
    transform: translateY(1px);  
    box-shadow: 0 5px 10px rgba(0, 240, 255, 0.2);  
}
```

```
@keyframes fadeIn {  
    from { opacity: 0; transform: translateY(20px); }  
    to { opacity: 1; transform: translateY(0); }  
}
```

```
@keyframes slideDown {  
    from { opacity: 0; transform: translateY(-15px); }  
    to { opacity: 1; transform: translateY(0); }  
}
```

```
@media (max-width: 480px) {  
    .container {  
        padding: 40px 25px;  
        margin: 20px;
```



```
}  
}
```

3. Valid_cs.js

```
document.addEventListener('DOMContentLoaded', function() {  
  try {  
    const form = document.getElementById('vehicleForm');  
    const regInput = document.getElementById('regNumber');  
    const errorDiv = document.getElementById('errorMessage');  
    const successDiv = document.getElementById('successMessage');  
    const rulesList = document.getElementById('rulesList');  
  
    if (!form || !regInput || !errorDiv || !successDiv || !rulesList) {  
      throw new Error("Required HTML elements are missing.");  
    }  
  
    const validationRules = [  
      "Must not be <strong>empty</strong>.",  
      "Exact length of <strong>10 characters</strong>.",  
      "First 2 chars: <strong>Uppercase Letters</strong> (State Code).",  
      "Next 2 chars: <strong>Digits</strong> (District Code).",  
      "Next 2 chars: <strong>Uppercase Letters</strong> (Series).",  
      "Last 4 chars: <strong>Digits</strong> (Vehicle Number)."  
    ];  
  
    function renderRules() {  
      rulesList.innerHTML = "";
```

```
validationRules.forEach(rule => {  
    const li = document.createElement('li');  
    li.innerHTML = rule;  
    rulesList.appendChild(li);  
});  
}  
  
renderRules();  
  
form.addEventListener('submit', function(event) {  
    try {  
        event.preventDefault();  
        const input = regInput.value.trim();  
        const pattern = /^[A-Z]{2}\d{2}[A-Z]{2}\d{4}$/;  
  
        errorDiv.style.display = 'none';  
        successDiv.style.display = 'none';  
  
        if (!input) {  
            showError("Field cannot be empty.");  
            return;  
        }  
  
        if (input.length !== 10) {  
            showError("Length must be exactly 10 characters.");  
            return;  
        }  
    }  
});
```

```
    if (pattern.test(input)) {  
        showSuccess("Valid Registration Number!");  
    } else {  
        showError("Format mismatch. Please check the rules above.");  
    }  
} catch (submitError) {  
    console.error("Validation error:", submitError);  
    showError("An unexpected error occurred.");  
}  
});
```

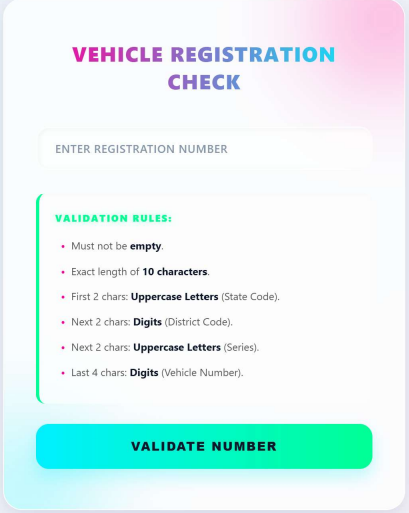
```
regInput.addEventListener('input', function() {  
    this.value = this.value.toUpperCase();  
});
```

```
function showError(message) {  
    errorDiv.textContent = message;  
    errorDiv.style.display = 'block';  
}
```

```
function showSuccess(message) {  
    successDiv.textContent = message;  
    successDiv.style.display = 'block';  
}
```

```
} catch (initError) {  
    console.error("Initialization failed:", initError);  
    alert("Failed to load form validation.");
```

```
}  
});  
Output
```



VEHICLE REGISTRATION CHECK

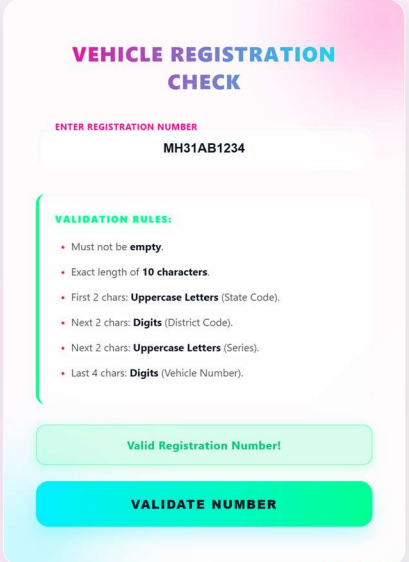
ENTER REGISTRATION NUMBER

VALIDATION RULES:

- Must not be **empty**.
- Exact length of **10 characters**.
- First 2 chars: **Uppercase Letters** (State Code).
- Next 2 chars: **Digits** (District Code).
- Next 2 chars: **Uppercase Letters** (Series).
- Last 4 chars: **Digits** (Vehicle Number).

VALIDATE NUMBER

CREATED BY: M Hussain Khan, PRN: 25070521505



VEHICLE REGISTRATION CHECK

ENTER REGISTRATION NUMBER

MH31AB1234

VALIDATION RULES:

- Must not be **empty**.
- Exact length of **10 characters**.
- First 2 chars: **Uppercase Letters** (State Code).
- Next 2 chars: **Digits** (District Code).
- Next 2 chars: **Uppercase Letters** (Series).
- Last 4 chars: **Digits** (Vehicle Number).

Valid Registration Number!

VALIDATE NUMBER

CREATED BY: M Hussain Khan, PRN: 25070521505

Result/Conclusion

The program was executed successfully. Different function types, variable scope, closures, and exception handling using try-catch were demonstrated, and the palindrome checker correctly identified whether the entered string was a palindrome.